



深圳大学
Shenzhen University

互联网编程

第10章

安全socket

深圳大学计算机与软件学院

保护Internet通信

■ 加密

- 明文、密钥、加密算法、密文
- 对称加密（秘密密钥）
- 非对称加密(公开密钥)
 - 用于身份认证
 - 用于消息完整性检查
- 二次加密
 - 自己私钥、他人公钥

■ 中间人攻击

- 通过可信第三方认证机构存储和验证公开密钥

■ Java安全socket扩展有四个包：

- [javax.net.ssl.SSLSocket](#)：安全socketAPI抽象类
- [javax.net.ssl.SSLSocketFactory](#)：创建安全Socket的抽象工厂
- [java.security.cert](#)：处理SSL需公开密钥证书类
- [com.sun.net.ssl](#)：具体实现类

创建安全客户端Socket

■ 示例

```
SSLSocketyFactory factory=(SSLSocketFactory)SSLSocketFactory.getDefault();
```

```
SSLSocket socket=null;
```

```
try{
```

```
    ■ socket=(SSLSocket) factory.creatSocket(host, port); //通常port=443
```

```
    ■ String[] supported=socket.getSupportCipherSuites();
```

```
    ■ Socket.setEnabledCipherSuites(supported);
```

```
    ■ // get I/O stream and read write as usual,
```

```
    ■ .....
```

```
}
```



创建安全客户端Socket

- 安全socket对象生产类SSLSocketFacotory
 - <http://docs.oracle.com/javase/8/docs/api/javax/net/SocketFactory.html>
 - static SocketFactory getDefault()
 - 继承自javax.net.SocketFactory
 - Socket createSocket()
 - abstract Socket createSocket(InetAddress host, int port)
 - abstract Socket createSocket(InetAddress address, int port, InetAddress localAddress, int localPort)
 - abstract Socket createSocket(String host, int port)
 - abstract Socket createSocket(String host, int port, InetAddress localHost, int localPort)
- SSLSocket
 - <http://docs.oracle.com/javase/8/docs/api/javax/net/ssl/SSLSocket.html>

使用安全Socket的客户端例子ex10-1 HTTPSCClient，程序使用安全socket实现连接一个安全服务器（美国邮政服务www.usps.com），发送简单的GET请求并响应。

使用安全Socket的客户端例子ex10-1 HTTPSClient，程序使用安全socket实现连接一个安全服务器（美国邮政服务www.usps.com），发送简单的GET请求并响应。

HTTPSClient.java

<https://www.usps.com/>

```
1 import java.io.*;
2 import javax.net.ssl.*;
3 public class HTTPSClient {
4
5     public static void main(String[] args) {
6
7         if (args.length == 0) {
8             System.out.println("Usage: java HTTPSClient2 host");
9             return;
10        }
11
12        int port = 443; // default https port
13        String host = args[0];
14
15        SSLSocketFactory factory
16            = (SSLSocketFactory) SSLSocketFactory.getDefault();
17        SSLSocket socket = null;
18        try {
19            socket = (SSLSocket) factory.createSocket(host, port);
20
21            // enable all the suites
22            String[] supported = socket.getSupportedCipherSuites();
23            socket.setEnabledCipherSuites(supported);
24
25            Writer out = new OutputStreamWriter(socket.getOutputStream(), "UTF-8");
26            // https requires the full URL in the GET line
27            out.write("GET http://" + host + "/ HTTP/1.1\r\n");
```

```
28 out.write("Host: " + host + "\r\n");
29 out.write("\r\n");
30 out.flush();
31
32 // read response
33 BufferedReader in = new BufferedReader(
34     new InputStreamReader(socket.getInputStream()));
35
36 // read the header
37 String s;
38 while (!(s = in.readLine()).equals("")) {
39     System.out.println(s);
40 }
41 System.out.println();
42
43 // read the length
44 String contentLength = in.readLine();
45 int length = Integer.MAX_VALUE;
46 try {
47     length = Integer.parseInt(contentLength.trim(), 16);
48 } catch (NumberFormatException ex) {
49     // This server doesn't send the content-length
50     // in the first line of the response body
51 }
52 System.out.println(contentLength);
53
54 int c;
```

实现连接
并响应。

```
55     int i = 0;
56     while ((c = in.read()) != -1 && i++ < length) {
57         System.out.write(c);
58     }
59
60     System.out.println();
61 } catch (IOException ex) {
62     System.err.println(ex);
63 } finally {
64     try {
65         if (socket != null) socket.close();
66     } catch (IOException e) {}
67 }
68 }
69 }
```



选择密码组

- SSLSocketFactory类中的方法：
- abstract String[] getSupportedCipherSuites()
 - ▣ 指出给定Socket上可用的算法组合
- abstract void setEnabledCipherSuites(String[] suites)
 - ▣ 修改客户端试图使用的密码组
- JDK1.7以上支持多个密码组，每个密码组名包括4个部分（协议、密钥交换算法、加密算法和校验和）：
- TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256
 - ▣ TLS(Transport Layer Security)与SSL(Secure Socket Layer)协议：TLS 是IETF接手SSL后制定的后续版本（国际标准）
 - ▣ ECDHE: Elliptic Curve Diffie-Hellman Exchange，基于ECC(Elliptic Curve Cryptography，椭圆曲线密码学，类似RSA)的密钥交换算法（在不安全的信道上安全的交换密钥）
 - ▣ ECDSA: 椭圆曲线数字签名算法
 - ▣ AES_128_CBC: Advanced Encryption Standard，DES的后续，128位，采用CBC（Cipher Block Chaining，密码分组链接模式）
 - ▣ SHA256: Secure Hash Algorithm，安全散列算法，类似MD5的消息摘要算法

客户端模式

- 安全Internet通信要求服务器证书认证自己，客户端通常不需要认证自己，若要求客户端需要认证自己，可使用：
 - `setUseClientMode(boolean mode)`
 - 确定socket是否需要在第一次握手时使用认证
- 如果服务端要求与它连接的所有客户端都需要认证，可以通过
 - `public abstract void setNeedClientAuth(boolean needsAuthentication)`来进行设置



创建安全服务器Socket

■ 步骤:

1. 使用keytool生成公开密钥和证书;
2. 请第三方机构认证证书;
3. 为算法创建一个SSLContext;
4. 为证书源创建TrustManagerFactory;
5. 为密钥类型创建KeyManagerFactory;
6. 为密钥和证书数据创建一个KeyStore对象;
7. 用密钥和证书填充KeyStore对象;
8. 用KeyStore及其口令短语初始化KeyManagerFactory
9. 用KeyManagerFactory中的密钥管理器、TrustManagerFactory中的信任管理器和一个随机源初始化上下文。

创建安全服务器Socket

■ SSLServerSocket

- <http://docs.oracle.com/javase/8/docs/api/javax/net/ssl/SSLServerSocket.html>
- 与SSLSocket类似，通过SSLServerSocketFactory创建

使用安全服务器Socket例子ex10-2 SecureOrderTaker，程序接受订单并显示在控制台输出。

需先使用JDK提供的keytool程序生成jnp4e.keys文件，keystore的口令为2andnotafnord
Keytool -genkey -alias ourstore -keystore jnp4e.keys



```
1 import java.io.*;
2 import java.net.*;
3 import java.security.*;
4 import java.security.cert.CertificateException;
5 import java.util.Arrays;
6 import javax.net.ssl.*;
7
8 public class SecureOrderTaker {
9
10     public final static int PORT = 7000;
11     public final static String algorithm = "SSL";
12
13     public static void main(String[] args) {
14         try {
15             SSLContext context = SSLContext.getInstance(algorithm);
16
17             // The reference implementation only supports X.509 keys
18             KeyManagerFactory kmf = KeyManagerFactory.getInstance("SunX509");
19
20             // Oracle's default kind of key store
21             KeyStore ks = KeyStore.getInstance("JKS");
22
23             // For security, every key store is encrypted with a
24             // passphrase that must be provided before we can load
25             // it from disk. The passphrase is stored as a char[] array
26             // so it can be wiped from memory quickly rather than
27             // waiting for a garbage collector.
```

```
28 char[] password = System.console().readPassword();
29 ks.load(new FileInputStream("jnp4e.keys"), password);
30 kmf.init(ks, password);
31 context.init(kmf.getKeyManagers(), null, null);
32
33 // wipe the password
34 Arrays.fill(password, '0');
35
36 SSLServerSocketFactory factory
37     = context.getServerSocketFactory();
38
39 SSLServerSocket server
40     = (SSLServerSocket) factory.createServerSocket(PORT);
41
42 // add anonymous (non-authenticated) cipher suites
43 String[] supported = server.getSupportedCipherSuites();
44 String[] anonCipherSuitesSupported = new String[supported.length];
45 int numAnonCipherSuitesSupported = 0;
46 for (int i = 0; i < supported.length; i++) {
47     if (supported[i].indexOf("_anon_") > 0) {
48         anonCipherSuitesSupported[numAnonCipherSuitesSupported++] =
49             supported[i];
50     }
51 }
52
53 String[] oldEnabled = server.getEnabledCipherSuites();
54 String[] newEnabled = new String[oldEnabled.length |
```

```
55     + numAnonCipherSuitesSupported];
56     System.arraycopy(oldEnabled, 0, newEnabled, 0, oldEnabled.length);
57     System.arraycopy(anonCipherSuitesSupported, 0, newEnabled,
58         oldEnabled.length, numAnonCipherSuitesSupported);
59
60     server.setEnabledCipherSuites(newEnabled);
61
62     // Now all the set up is complete and we can focus
63     // on the actual communication.
64     while (true) {
65         // This socket will be secure,
66         // but there's no indication of that in the code!
67         try (Socket theConnection = server.accept()) {
68             InputStream in = theConnection.getInputStream();
69             int c;
70             while ((c = in.read()) != -1) {
71                 System.out.write(c);
72             }
73         } catch (IOException ex) {
74             ex.printStackTrace();
75         }
76     }
77 } catch (IOException | KeyManagementException
78     | KeyStoreException | NoSuchAlgorithmException
79     | CertificateException | UnrecoverableKeyException ex) {
80     ex.printStackTrace();
81 }
```

配置SSLServerSocket

- SSLServerSocket类也提供了相应的方法支持下面的设置。
- 选择密码组
 - `Public abstract String[] getSupportedSuites()`
 - `Public abstract String[] getEnabledCipherSuites()`
 - `Public abstract void setEnabledCipherSuites(String[] suites)`
- 会话管理
 - `Public abstract void setEnableSessionCreation(Boolean allowSessions)`
 - `Public abstract Boolean getEnableSessionCreation()`
- 客户端模式
 - 确定和指定是否要求客户端向服务器认证自己。
 - `Public abstract void setNeedClientAuth(Boolean flag)`
 - `Public abstract Boolean getNeedClientAuth()`



谢谢

